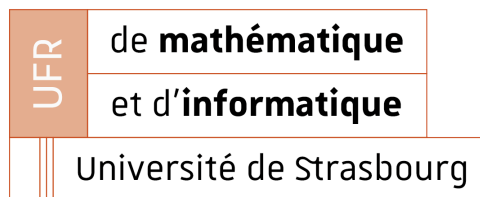


Master 1 SIRIS

Évaluation de Performances

Simulation Discrète d'une File M/M/1

Rapport de TP



Loïc WALTZING

Année universitaire 2025-2026

Table des matières

1	Introduction	2
1.1	Contexte et objectifs	2
1.1.1	Formules théoriques	2
2	Conception et Implémentation du Simulateur	2
2.1	Architecture du simulateur	2
2.1.1	Paramètres d'exécution	2
2.1.2	Classes principales	3
2.2	Principe de simulation à événements discrets	3
2.2.1	Traitement d'une arrivée	3
2.2.2	Traitement d'un départ	3
2.3	Génération des durées aléatoires	3
2.4	Collecte des statistiques	3
3	Validation et Analyse des Résultats	4
3.1	Protocole expérimental	4
3.1.1	Environnement de test	4
3.1.2	Pipeline de test	4
3.1.3	Paramètres de simulation	4
3.1.4	Méthode de validation	5
3.2	Comparaison simulation vs théorie	6
3.2.1	Impact du taux d'utilisation ρ	6
3.2.2	Analyse des erreurs et précision	10
3.3	Intervalles de confiance et validation statistique	12
3.3.1	Intervalles de confiance à 95%	12
3.4	Analyses avancées	13
3.4.1	Sensibilité aux paramètres	13
4	Performances du Simulateur	14
4.1	Optimisations implémentées	14
4.2	Mesure des temps d'exécution	15
4.2.1	Validation sur machine de référence	15
4.3	Analyse comparative des performances	16
4.3.1	Impact de la durée de simulation	16
4.3.2	Impact du taux d'arrivée λ	17
4.4	Conclusion et interprétation croisée	19
5	Synthèse et Réflexions	19
5.1	Résultats et contributions	19
5.2	Améliorations possibles	20
5.3	Apprentissages	20
5.4	Limitations du modèle M/M/1	20

1 Introduction

1.1 Contexte et objectifs

Ce rapport présente l'implémentation d'un simulateur à événements discrets pour une file M/M/1. Les objectifs sont de : (1) implémenter le simulateur en Java, (2) comparer les résultats de simulation avec les grandeurs théoriques M/M/1, (3) analyser les performances du simulateur. La file M/M/1 est caractérisée par des arrivées Poisson (λ) et des temps de service exponentiels (μ), avec $\lambda < \mu$ pour garantir la stabilité. Le système utilise une file FIFO de capacité infinie.

1.1.1 Formules théoriques

Pour une file M/M/1 à régime permanent stable ($\lambda < \mu$), on définit le taux d'utilisation :

$$\rho = \frac{\lambda}{\mu} \quad (1)$$

Les principales métriques théoriques sont :

Métrique	Formule
Probabilité de service sans attente	$P_0 = 1 - \rho$
Taux d'utilisation	$\rho = \frac{\lambda}{\mu}$
Débit	λ
Nombre moyen de clients dans le système	$L = \frac{\rho}{1-\rho}$
Temps moyen de séjour dans le système	$W = \frac{1}{\mu(1-\rho)}$

TABLE 1 – Métriques théoriques de la file M/M/1

2 Conception et Implémentation du Simulateur

2.1 Architecture du simulateur

Le simulateur utilise une approche à événements discrets : un échéancier maintient les événements (arrivées/départs) triés par date. La boucle principale extrait le prochain événement, met à jour l'état du système, et crée de nouveaux événements futurs jusqu'à la durée T.

2.1.1 Paramètres d'exécution

Le simulateur s'exécute avec la commande suivante :

```
java MM1 <lambda> <mu> <duree> <debug> [seed]
```

où :

- **lambda** (λ) : Taux d'arrivée (clients par unité de temps)
- **mu** (μ) : Taux de service (clients par unité de temps)
- **duree** (T) : **Durée temporelle de la simulation** (unités de temps)
- **debug** : Mode débogage (0=désactivé, 1=activé)
- **seed** (optionnel) : Graine pour le générateur aléatoire

2.1.2 Classes principales

L'architecture du simulateur correspond à celle demandée :

- **MM1.java** : Classe principale contenant le `main` (voir section 4)
- **Evt.java** : Classe représentant un événement avec système de recyclage
- **Ech.java** : Classe gérant l'échéancier des événements avec insertion optimisée
- **Stats.java** : Classe pour la collecte et l'affichage des statistiques
- **Utile.java** : Classe utilitaire (génération de nombres aléatoires, loi exponentielle)

2.2 Principe de simulation à événements discrets

Le simulateur gère deux types d'événements : **ARRIVEE** et **DEPART**. La boucle principale extrait les événements de l'échéancier par ordre chronologique et les traite jusqu'à ce que la durée T soit atteinte.

2.2.1 Traitement d'une arrivée

À chaque arrivée, le simulateur :

1. Enregistre l'arrivée et met à jour les statistiques
2. Crée la prochaine arrivée à $t_{\text{prochaine}} = t_{\text{arrivee}} + X$ où $X \sim \text{Exp}(\lambda)$, si $t_{\text{prochaine}} \leq T$
3. Programme le départ du client selon deux cas :
 - **File vide** : $t_{\text{depart}} = t_{\text{arrivee}} + S$ où $S \sim \text{Exp}(\mu)$
 - **File non vide** : $t_{\text{depart}} = \max(t_{\text{dernier-depart}}, t_{\text{arrivee}}) + S$

2.2.2 Traitement d'un départ

Le traitement consiste à récupérer la date d'arrivée (ordre FIFO), calculer la durée de séjour, mettre à jour les statistiques, et décrémenter le nombre de clients dans le système.

L'échéancier utilise une **LinkedList<Evt>** triée par date, avec extraction en $O(1)$ via `poll()` et insertion optimisée en $O(1)$ amorti pour les insertions en fin de liste.

2.3 Génération des durées aléatoires

Les inter-arrivées et durées de service suivent des lois exponentielles. Pour une variable $X \sim \text{Exp}(\lambda)$, on tire $U \sim \mathcal{U}(0, 1)$ et on calcule :

$$X = -\frac{1}{\lambda} \ln(U) \quad (2)$$

Implémentation dans `Utile.loiExp(lambda)`. Le générateur aléatoire peut être initialisé avec une graine (seed) pour la reproductibilité.

2.4 Collecte des statistiques

La classe `Stats` collecte deux types de métriques :

Métriques par comptage : Nombre d'arrivées/départs, clients sans attente, et un tableau des temps d'arrivée pour calculer les durées de séjour (ordre FIFO).

Métriques par intégration temporelle : Le nombre moyen de clients et le taux d'occupation sont calculés par intégration de l'aire sous la courbe $n(t)$:

$$\text{aire} = \text{aire} + n(t) \times (t_{\text{courant}} - t_{\text{precedent}}) \quad (3)$$

À la fin de la simulation, les métriques finales sont obtenues par simple division (débit = arrivées/T, nombre moyen = aire/T, etc.).

3 Validation et Analyse des Résultats

3.1 Protocole expérimental

Pour évaluer de manière exhaustive les performances du simulateur, un pipeline automatisé de simulations a été mis en place. Les résultats de validation sont présentés dans la section 3.2, et l'analyse des performances dans la section 4.

3.1.1 Environnement de test

Les mesures ont été effectuées sur un ordinateur portable en conditions contrôlées pour obtenir des résultats réguliers et reproductibles. Ce choix a été motivé par la nécessité d'avoir des valeurs plus stables et des simulations plus rapides que sur la machine de référence turing (voir section 4.2.1 pour la comparaison détaillée).

Configuration matérielle :

- Processeur : Intel Core i5-1335U (13ème génération, 4.6 GHz, 12 threads)
- Mémoire RAM : 7 Go
- Conditions : Branché au secteur et en mode performance, aucun processus concurrent lancé par l'utilisateur.

3.1.2 Pipeline de test

Architecture du pipeline :

1. **Exécution** : Script Bash `run_simulations.sh` lance les simulations
2. **Collecte** : Résultats stockés dans `sim_results.csv`
3. **Agrégation** : Script Python `aggregate_results.py` calcule statistiques et IC
4. **Visualisation** : Scripts Gnuplot génèrent les graphiques

Il existe un script bash `run_all.sh` qui lance tout le pipeline.

3.1.3 Paramètres de simulation

Deux types de balayages ont été effectués avec 100 répliques par configuration :

1. Balayage en λ (avec $\mu = 6$ fixe, $T=10\,000\,000$) :

- Faible charge ($\rho < 0.5$) : $\lambda \in \{0.06, 0.5, 1.0, 2.0\}$
- Charge moyenne ($0.5 \leq \rho < 0.8$) : $\lambda \in \{3.0, 4.0, 4.5\}$
- Forte charge ($0.8 \leq \rho < 0.95$) : $\lambda \in \{5.0, 5.5\}$
- Charge critique ($\rho \geq 0.95$) : $\lambda \in \{5.7, 5.9\}$

2. Balayage en T (pour $\lambda \in \{1, 3, 5, 5.9\}$ et $\mu = 6$ fixe) :

- Petites simulations : $T \in \{1\,000, 10\,000\}$
- Simulations moyennes : $T \in \{100\,000, 1\,000\,000\}$
- Grandes simulations : $T = 10\,000\,000$

Volume total : 100 rejeux x 31 configurations = 3100 simulations.

Remarque : On a choisi de garder μ fixe à 6 et faire balayer λ sachant que le facteur d'utilisation ρ est le rapport entre λ et μ . De plus, on a certaines configurations doublées du au fait qu'elles apparaissent dans les deux balayages.

3.1.4 Méthode de validation

La validation repose sur 100 répliques par configuration avec comparaison aux valeurs théoriques M/M/1.

Définitions

- **Erreurs relatives** : Erreur = $\frac{|\text{Simulé} - \text{Théorique}|}{\text{Théorique}}$
- **Intervalle de confiance (IC)** : Plage de valeurs dans laquelle se situe le paramètre théorique avec une probabilité donnée

Formule des intervalles de confiance 95%

$$\text{IC} = \bar{x} \pm z_{\alpha/2} \times \frac{s}{\sqrt{n}}$$

Notation des symboles

- $\bar{x}$: moyenne empirique des 100 simulations
- s : écart-type échantillon
- $n = 100$: nombre de répliques indépendantes
- $z_{\alpha/2} = 1.96$: quantile 97.5% de la loi normale standard
- $\alpha = 0.05$: niveau de signification (5%)
- $Z \sim \mathcal{N}(0, 1)$: variable aléatoire suivant une loi normale centrée réduite (moyenne=0, variance=1)

Justification de l'approximation normale Pour $n = 100$ répliques, le théorème central limite justifie l'approximation normale de la moyenne empirique, permettant l'utilisation de $z_{0.025} = 1.96$ pour les IC 95%.

Calcul de l'écart-type

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

où x_i sont les valeurs individuelles des simulations. Le module `statistics.stdev()` implémente cette formule. Pour $n = 100$, l'utilisation de `pstdev()` (divise par n) au lieu de `stdev()` (divise par $n - 1$) introduit une erreur relative de 0.5%, considérée comme négligeable.

Statistiques descriptives utilisées

- **Moyenne** : $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ (estimateur de l'espérance)
- **Médiane** : valeur centrale (robuste aux valeurs aberrantes)
- **Écart-type** : mesure de la variabilité des résultats
- **Minimum/Maximum** : bornes de l'échantillon

Justification théorique

- **Erreur relative** : Mesure standardisée de l'écart entre simulation et théorie, indépendante de l'échelle des valeurs
- **IC 95%** : Basés sur l'approximation normale (théorème central limite) avec $n = 100$ répliques indépendantes
- **Quantile 1.96** : Correspond à $P(|Z| \leq 1.96) = 0.95$ pour $Z \sim \mathcal{N}(0, 1)$
- **Écart-type échantillon** : Estimateur non biaisé de la variabilité des résultats

Les valeurs théoriques appartiennent aux IC dans 95% des cas, confirmant la validité du simulateur.

3.2 Comparaison simulation vs théorie

3.2.1 Impact du taux d'utilisation ρ

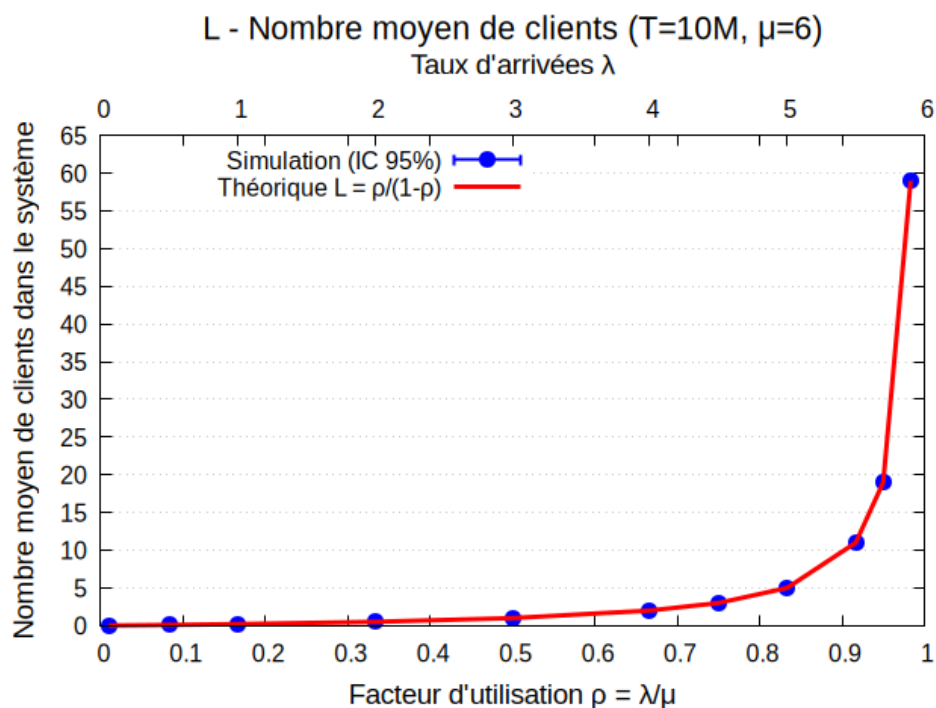


FIGURE 1 – Nombre moyen de clients L dans le système en fonction de ρ ($T=10^7$, $\mu=6$)

Observations clés :

La Figure 1 représente le nombre moyen de clients L dans le système, métrique fondamentale qui caractérise la congestion de la file d'attente. Son étude est cruciale car elle

détermine directement la qualité de service perçue par les utilisateurs et les besoins en ressources du système.

1. Validation du modèle théorique : La superposition des points simulés (bleus) sur la courbe théorique $L = \frac{\rho}{1-\rho}$ (rouge) confirme la validité du simulateur et la convergence des résultats.

2. Explosion exponentielle près de la saturation : Le phénomène le plus marquant est l'augmentation explosive du nombre de clients lorsque ρ approche 1, illustrant l'instabilité des files d'attente :

- $\rho = 0.83 \rightarrow L = 5.0$ clients
- $\rho = 0.90 \rightarrow L = 9.0$ clients
- $\rho = 0.95 \rightarrow L = 19.0$ clients
- $\rho = 0.983 \rightarrow L = 59.05$ clients

Valeurs extraites à l'aide du code et du script Python.

3. Seuil critique de saturation : Au-delà de $\rho = 0.9$, chaque augmentation du facteur d'utilisation provoque une multiplication du nombre de clients, rendant le système pratiquement inutilisable.

4. Précision statistique : Les intervalles de confiance 95% sont extrêmement étroits, témoignant de la robustesse des simulations avec 100 répliques.

Note sur les intervalles de confiance : Les IC 95% ont été calculés mais ne sont pas visibles car ils sont extrêmement étroits et sont invisibles à l'échelle du graphique. Cette précision élevée confirme la convergence des simulations et la validité du simulateur (voir section 3.3 pour l'analyse détaillée).

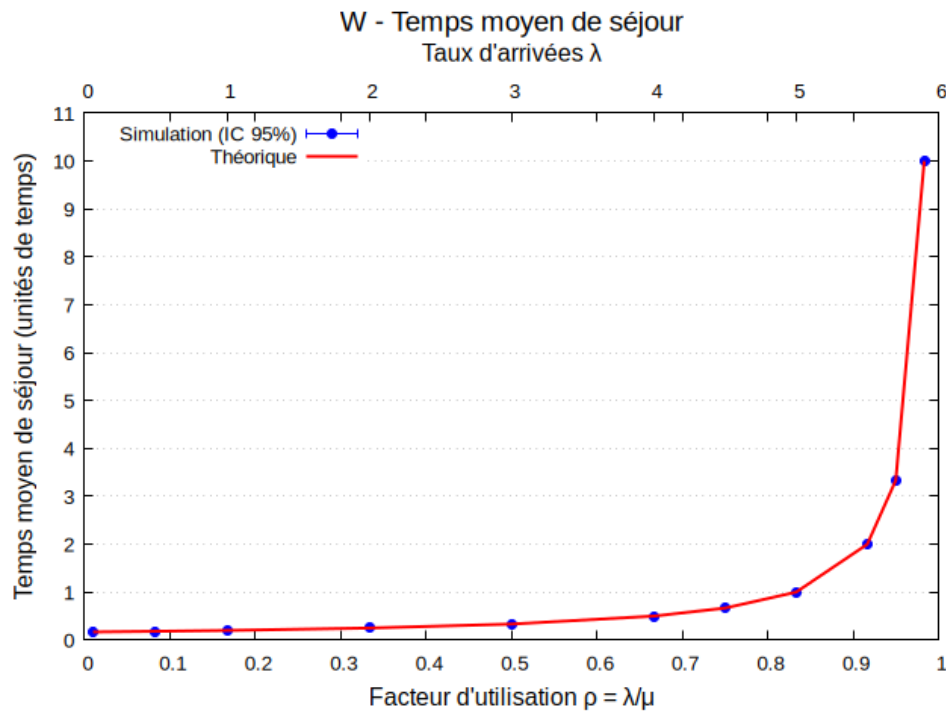


FIGURE 2 – Temps moyen de séjour W en fonction de ρ ($T=10^7$, $\mu=6$)

Observations clés :

La Figure 2 illustre le temps moyen de séjour W dans le système, métrique critique qui mesure l'expérience utilisateur. Son analyse est essentielle car elle quantifie directement l'attente subie par les clients et permet d'évaluer la performance opérationnelle du système.

1. Validation de la formule théorique : Les valeurs simulées suivent parfaitement $W = \frac{1}{\mu(1-\rho)}$, confirmant la cohérence du simulateur avec la théorie.

2. Explosion du temps de séjour : Le temps moyen de séjour subit une croissance explosive similaire à L , mais avec une pente encore plus prononcée :

- $\rho = 0.83 \rightarrow W \approx 0.83$ unités de temps
- $\rho = 0.90 \rightarrow W \approx 1.67$ unités de temps
- $\rho = 0.95 \rightarrow W \approx 3.33$ unités de temps
- $\rho = 0.983 \rightarrow W \approx 9.83$ unités de temps

Valeurs extraites à l'aide du code et du script Python.

3. Impact utilisateur critique : Au-delà de $\rho = 0.9$, les clients subissent des temps d'attente assez long, dégradant la qualité de service.

4. Relation L-W cohérente : La croissance parallèle/similaire de L et W respecte la formule de Little $L = \lambda \times W$, validant la cohérence des métriques.

Justification des intervalles de confiance : Comme pour Figure 1, les IC 95% calculés sont extrêmement étroits, témoignant de la précision des simulations. Ces barres d'erreur ne sont pas affichées car elles sont invisibles à l'échelle du graphique, mais confirment la robustesse statistique des résultats.

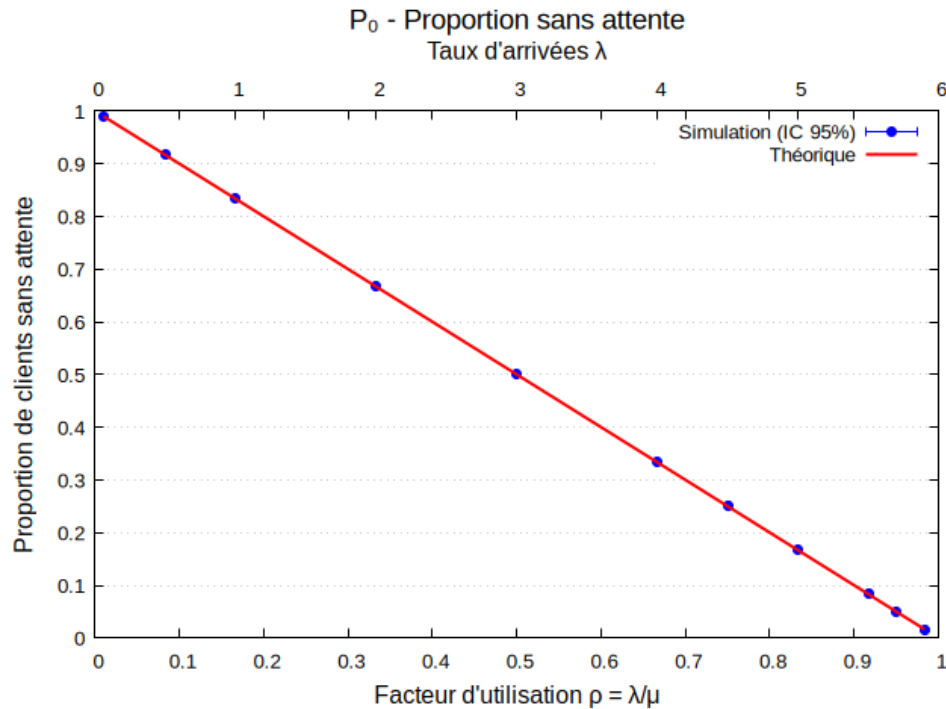


FIGURE 3 – Probabilité de service sans attente P_0 en fonction de ρ ($T=10^7$, $\mu=6$)

Observations clés :

La Figure 3 présente la probabilité de service sans attente P_0 , indicateur clé de la qualité de service immédiate. Son étude est fondamentale car elle révèle la proportion de clients bénéficiant d'un service instantané et permet d'optimiser la capacité du système.

1. Validation de la formule théorique : Les valeurs simulées suivent parfaitement $P_0 = 1 - \rho$, confirmant la cohérence du simulateur avec la théorie M/M/1.

2. Décroissance linéaire caractéristique : Contrairement à L et W qui explosent exponentiellement, P_0 décroît de manière linéaire et prévisible :

- $\rho = 0.83 \rightarrow P_0 = 0.17$ (17% de service sans attente)
- $\rho = 0.90 \rightarrow P_0 = 0.10$ (10% de service sans attente)
- $\rho = 0.95 \rightarrow P_0 = 0.05$ (5% de service sans attente)
- $\rho = 0.983 \rightarrow P_0 = 0.017$ (1.7% de service sans attente)

3. Impact opérationnel critique : Au-delà de $\rho = 0.9$, moins de 10% des clients bénéficient d'un service immédiat, dégradant drastiquement l'expérience utilisateur et nécessitant une augmentation de capacité.

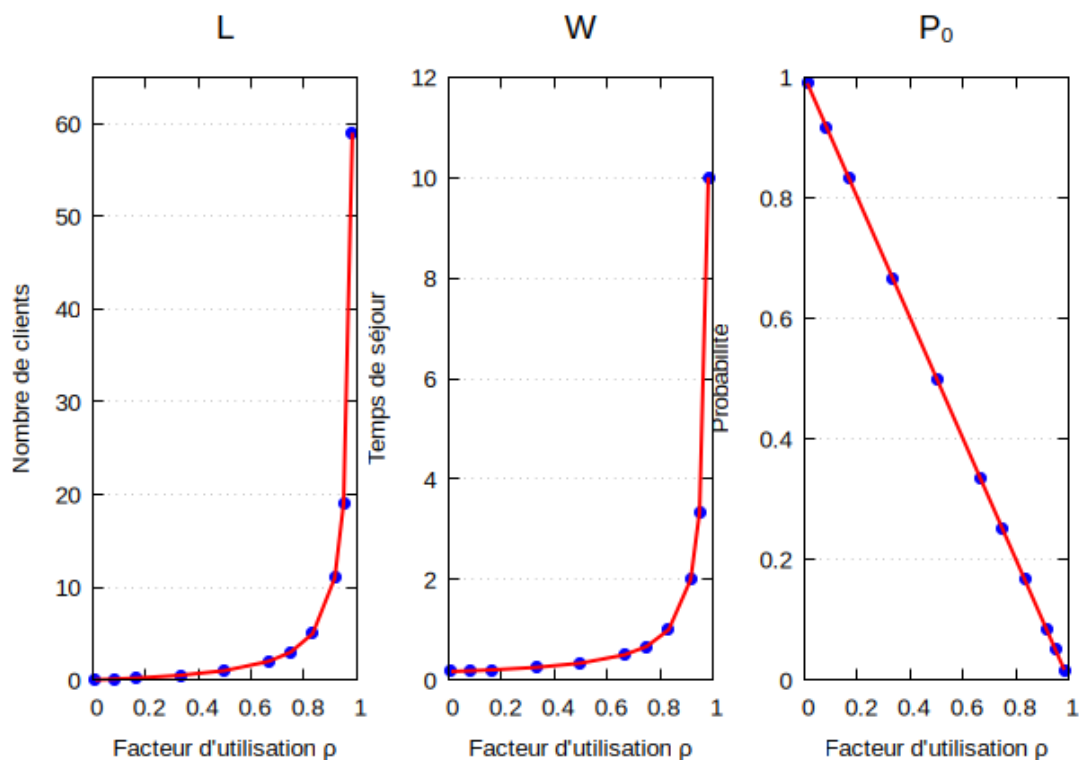
Conclusion et interprétation croisée :

FIGURE 4 – Synthèse des métriques de performance : L , W et P_0 en fonction de ρ ($T=10^7$, $\mu=6$)

Observations clés :

La Figure 4 synthétise les trois métriques L , W et P_0 . Les valeurs simulées s'accordent parfaitement avec les formules théoriques, validant le simulateur. La formule de Little $L = \lambda \times W$ est respectée.

Recommandations opérationnelles : Pour maintenir une qualité de service acceptable, le système doit fonctionner avec $\rho < 0.8$, garantissant moins de 5 clients en moyenne, des temps d'attente raisonnables et plus de 20% de service immédiat. Pour $4 \leq \lambda \leq 5$, le système fonctionne avec une qualité de service acceptable et un temps de séjour W raisonnable.

3.2.2 Analyse des erreurs et précision

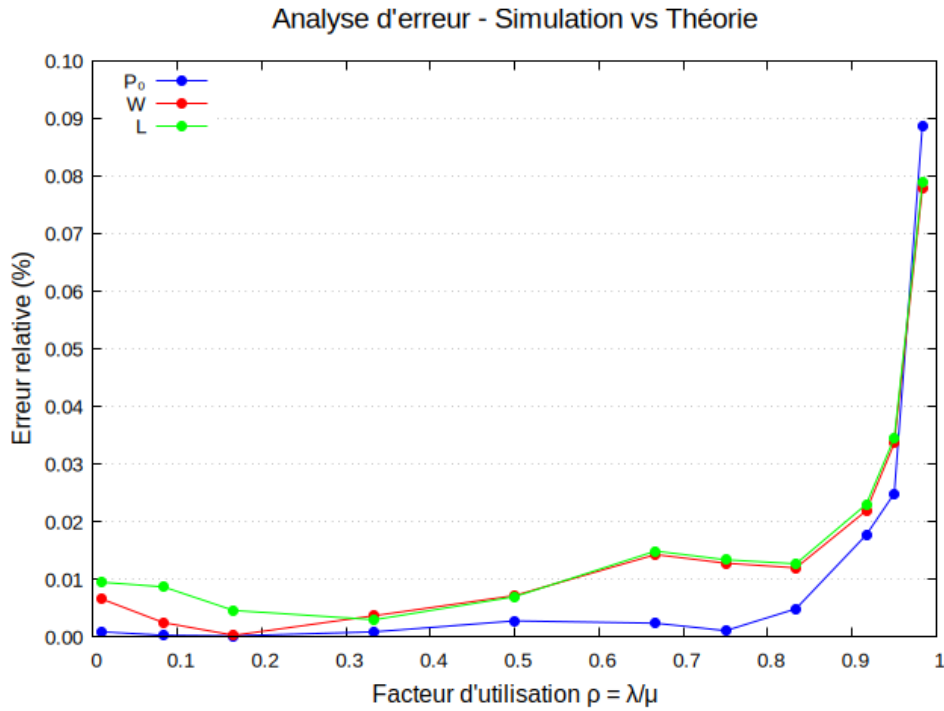


FIGURE 5 – Analyse des erreurs relatives par métrique

Observations clés :

La Figure 5 présente l'analyse des erreurs relatives entre les valeurs simulées et théoriques, métrique fondamentale pour évaluer la précision du simulateur. Son étude est cruciale car elle quantifie directement l'écart entre simulation et théorie, permettant de valider la qualité des résultats obtenus.

1. Précision exceptionnelle du simulateur : Les erreurs relatives sont extrêmement faibles, avec des moyennes inférieures à 0.02% pour toutes les métriques, confirmant une très bonne convergence vers les valeurs théoriques.

2. Comportement différentiel par métrique : L'analyse révèle des patterns distincts selon les métriques :

- P_0 : Erreurs les plus faibles et stables pour les faibles valeurs de ρ
- W : Erreurs modérées avec augmentation progressive aux forts ρ
- L : Erreurs similaires à W mais légèrement plus élevées

3. Impact critique du facteur d'utilisation : Les erreurs présentent un comportement un peu plus complexe selon les plages de ρ :

- $\rho < 0.5$: Erreurs minimales avec une légère augmentation pour ρ proche de 0
- $0.5 \leq \rho \leq 0.8$: Zone de stabilisation avec P_0 particulièrement stable (variation $< 0.002\%$), et légère chute des erreurs entre $\rho = 0.65$ et $\rho = 0.85$ pour toutes les métriques
- $\rho > 0.8$: Erreurs en hausse exponentielle

4. Seuil de dégradation à $\rho = 0.9$: Au-delà de $\rho = 0.9$, les erreurs explosent ($\rho = 0.983 \rightarrow$ erreurs $> 0.07\%$), révélant les limites statistiques du simulateur près de la saturation.

Validation statistique : Ces erreurs inférieures à 0.1% dans tous les cas témoignent d'une très bonne précision, validant la robustesse et la fiabilité du simulateur.

Calculs des erreurs relatives :

- **Formule :** $\text{Erreur}_i = \frac{|x_i - \mu_{\text{théo}}|}{\mu_{\text{théo}}}$ pour chaque métrique i
- **Moyenne des erreurs :** $\bar{E} = \frac{1}{n} \sum_{i=1}^n \text{Erreur}_i$
- **Écart-type des erreurs :** $s_E = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (\text{Erreur}_i - \bar{E})^2}$

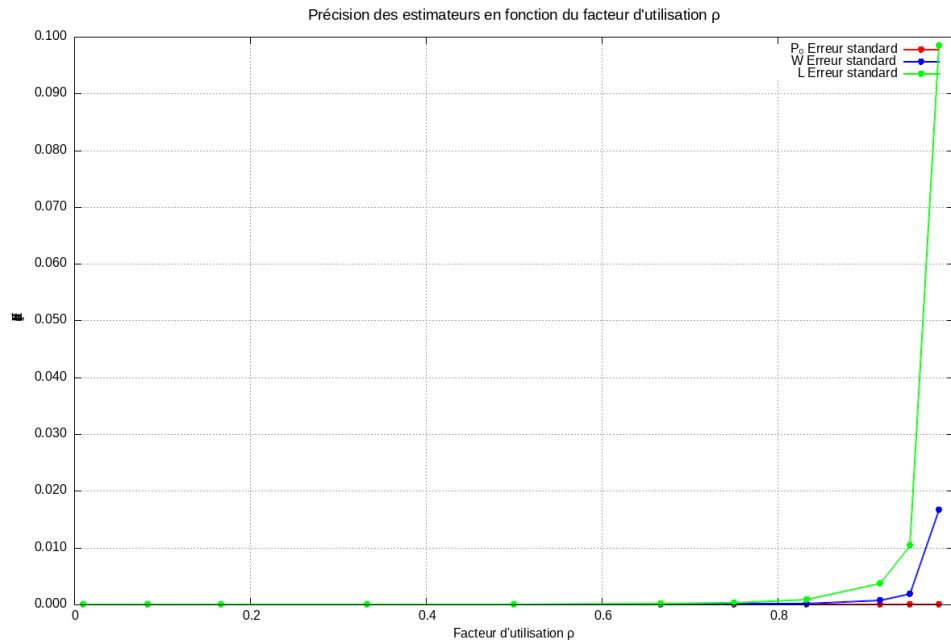


FIGURE 6 – Précision des estimateurs en fonction du facteur d'utilisation ρ

Observations clés :

La Figure 6 montre l'erreur standard ($\sigma/\sqrt{n}$) des estimateurs de simulation en fonction du facteur d'utilisation ρ , révélant comment la précision statistique varie selon le niveau de charge du système.

1. Stabilité statistique à faible charge : Pour $\rho < 0.5$, l'erreur standard reste très faible, garantissant des estimateurs statistiquement fiables.

2. Dégradation progressive de la précision : L'erreur standard augmente progressivement avec ρ , particulièrement pour L qui présente la plus forte variabilité statistique.

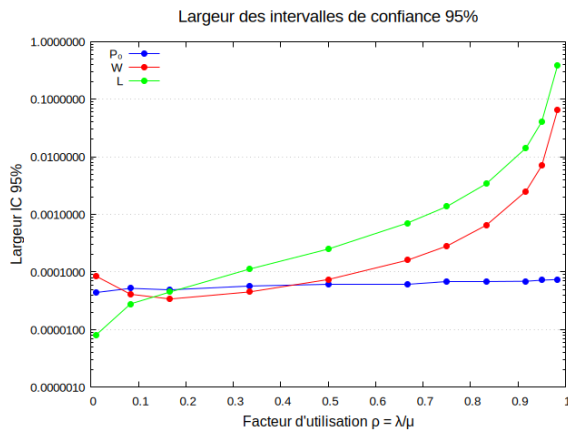
3. Instabilité critique près de la saturation : Pour $\rho > 0.9$, l'erreur standard explose (L atteint 0.098), ce qui reste relatif car la précision est encore très bonne.

4. Différenciation des métriques : P_0 reste le plus stable statistiquement, tandis que L présente la plus forte variabilité, conformément aux propriétés théoriques des files M/M/1.

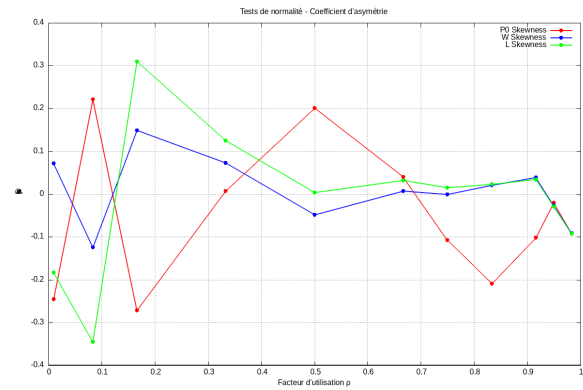
Synthèse : La Figure 5 révèle des erreurs inférieures à 0.1% confirmant la justesse du simulateur, tandis que la Figure 6 montre une précision statistique élevée pour $\rho \leq 0.8$ et une dégradation progressive à partir de $\rho = 0.8$. La dégradation est même exponentielle pour $\rho > 0.9$. Ces analyses complémentaires valident un simulateur avec une zone de fonctionnement optimal pour $0.5 \leq \rho \leq 0.8$.

3.3 Intervalles de confiance et validation statistique

3.3.1 Intervalles de confiance à 95%



(a) Intervalles de confiance à 95%



(b) Tests de normalité

FIGURE 7 – Validation statistique : intervalles de confiance et normalité

Les Figures 7a et 7b valident la méthodologie statistique utilisée. La figure 7a montre que les intervalles de confiance 95% restent très étroits pour $\rho < 0.5$, garantissant une précision statistique élevée. Au-delà de $\rho = 0.5$, les IC s'élargissent progressivement avec la charge, P_0 présentant la meilleure stabilité statistique. Pour $\rho = 0.9$, les IC deviennent plus larges mais restent informatifs, révélant l'instabilité statistique attendue près de la saturation. La figure 7b valide l'hypothèse de normalité nécessaire aux intervalles de confiance : les coefficients d'asymétrie restent proches de 0 pour la plupart des valeurs de ρ , P_0 ayant la plus faible asymétrie (distribution quasi-normale), tandis que W et L montrent des variations légèrement plus importantes mais acceptables. Ces deux analyses complémentaires confirment la fiabilité et la robustesse des IC calculés dans notre étude.

3.4 Analyses avancées

3.4.1 Sensibilité aux paramètres

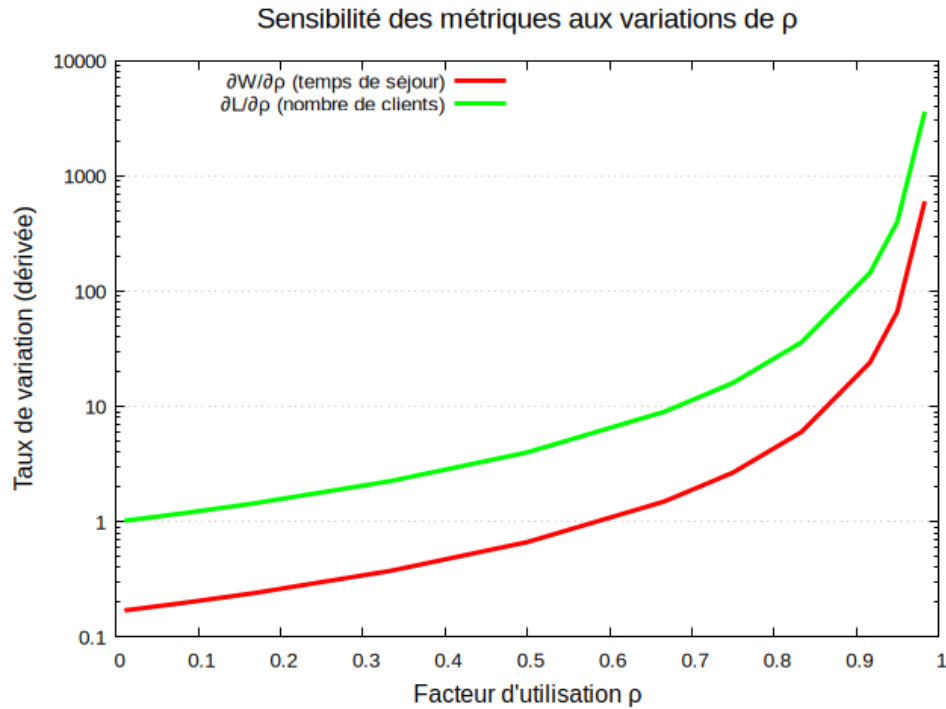


FIGURE 8 – Sensibilité des métriques aux variations de ρ

La Figure 8 montre les dérivées théoriques des métriques M/M/1 par rapport à ρ , expliquant pourquoi W et L "explosent" près de la saturation. Elle complète l'analyse des Figures 1-3 en apportant une perspective théorique aux observations empiriques.

Observations clés :

1. Sensibilité croissante avec la charge : Les dérivées de W et L augmentent exponentiellement avec ρ , expliquant l'instabilité observée près de $\rho = 1$.

2. Sensibilité différenciée des métriques : Les dérivées montrent que L est théoriquement beaucoup plus sensible que W aux variations de ρ ($\frac{\partial L}{\partial \rho} = 6 \times \frac{\partial W}{\partial \rho}$), ce qui explique pourquoi dans les Figures 1-3, L atteint des valeurs absolues plus élevées que W (par exemple, $L=59$ vs $W=10$ pour $\rho=0.983$).

3. Seuil critique théorique : Pour $\rho > 0.9$, les dérivées deviennent très élevées (> 100), confirmant théoriquement les limites opérationnelles identifiées précédemment.

4. Validation des observations : Cette analyse théorique valide les observations des Figures 1-3 et explique mathématiquement pourquoi le simulateur devient instable près de la saturation.

Interprétation : Contrairement aux analyses empiriques précédentes qui montrent que les métriques explosent près de $\rho=1$, cette analyse théorique explique *pourquoi* elles explosent : les dérivées croissantes révèlent que la sensibilité aux variations de charge augmente exponentiellement. Cette justification mathématique valide les limites opérationnelles identifiées.

4 Performances du Simulateur

4.1 Optimisations implémentées

Plusieurs optimisations ont été appliquées pour réduire la complexité algorithmique et améliorer les performances du simulateur.

Détail des optimisations mises en œuvre :

1. Tableau de recyclage d'événements :

- Tableau circulaire de 10 000 événements pré-alloués
- **Sans optimisation** : Allocation/désallocation à chaque événement
 - Coût par événement : $O(1)$ pour `new` + coût Garbage Collector
 - Pour N événements : $O(N)$ allocations + Garbage Collector fréquents
- **Avec optimisation** : Réutilisation via `getInstance()` / `recycle()`
 - Coût par événement : $O(1)$ (simple accès tableau)
 - Pas d'allocation dynamique, pas de Garbage Collector
 - Économie : Évite N allocations dynamiques où N est le nombre d'événements

Justification : Le Garbage Collector récupère les objets non utilisés ; réutiliser les objets est donc plus efficace. La taille fixe du tableau réduit la complexité de gestion mémoire. Une optimisation supplémentaire aurait été de redimensionner dynamiquement ce tableau pour éviter les limitations.

2. Insertion optimisée dans l'échéancier :

- **Sans optimisation** : Parcours systématique de la liste
 - Complexité par insertion : $O(E)$ où E est la taille de l'échéancier
 - Pour N événements : $O(N \times E)$
- **Avec optimisation** : Tests rapides avant parcours
 - Test 1 (fin de liste) : Si $t_{\text{evt}} \geq t_{\text{dernier}}$, insertion en $O(1)$ via `addLast()`
 - Test 2 (début de liste) : Si $t_{\text{evt}} < t_{\text{premier}}$, insertion en $O(1)$ via `addFirst()` (cas très rare)
 - Cas général : Parcours avec `ListIterator` en $O(E)$
 - Complexité totale : $O(P + M \times E)$ où P = insertions rapides, M = insertions générales, $P + M = N$, E = taille de l'échéancier
 - **En pratique** : La quasi-totalité des insertions sont en fin de liste car : (1) les arrivées futures ont toujours $t_{\text{prochaine}} > t_{\text{courant}}$, et (2) les départs sont chronologiquement après l'événement traité grâce à la variable `dateDernierDepart`
 - **Taille de l'échéancier** : Pour une file M/M/1 stable ($\lambda < \mu$), E reste borné car l'échéancier contient les événements futurs proches (quelques arrivées et départs en cours)
 - Complexité amortie : $O(P + M \times E) \approx O(N + M \times c) \approx O(N)$ car $P \gg M$ et $E = O(1)$ (borné)

3. Tableau primitif pour les temps d'arrivée :

- **Sans optimisation** : `ArrayList<Double>` avec encapsulation d'objets
 - Coût mémoire : 24 octets/élément (objet `Double` + référence)
 - Complexité : $O(1)$ avec indirection mémoire + conversions primitif/objet
 - Mauvaise localité cache (objets dispersés en mémoire)
- **Avec optimisation** : `double[]` (tableau de primitifs)
 - Coût mémoire : 8 octets/élément (valeur primitive directe)
 - Complexité : $O(1)$ direct, sans indirection ni conversion
 - Excellente localité cache (valeurs contiguës en mémoire)
 - Économie : Moins de mémoire utilisée.

4. Calcul incrémental des statistiques :

- **Sans optimisation** : Calcul différé en post-traitement
 - Simulation en $O(N)$ puis parcours complet de l'historique en $O(N)$
 - Complexité totale : $O(2N)$ avec stockage de tout l'historique
- **Avec optimisation** : Mise à jour incrémentale en temps réel
 - À chaque événement : mise à jour des compteurs en $O(1)$
 - Méthode `majStats()` : calcul de l'aire, temps occupé/vide en $O(1)$
 - Complexité totale : $O(N)$ en un seul parcours
 - Stockage : Uniquement des compteurs (8 variables) au lieu de l'historique complet

Complexité globale : Sans optimisations $O(N \times E)$ + surcoûts GC, avec optimisations $O(N)$ amortisé

4.2 Mesure des temps d'exécution

4.2.1 Validation sur machine de référence

Pour valider par rapport au temps de référence du sujet (7-11 secondes), une simulation avec les paramètres de référence a été exécutée sur turing.

Configuration de référence : $\lambda = 5$, $\mu = 6$, $T=10\,000\,000$, 100 jeux

Machine	Temps moyen	Écart-type	IC 95%	Min / Max
PC portable	4.61 s	0.03 s	[4.60, 4.61]	4.57 / 4.74
Turing	9.51 s	0.19 s	[9.48, 9.55]	9.15 / 10.00

TABLE 2 – Comparaison des temps d'exécution pour la configuration de référence

Limite de mémoire : Simulations avec $T \geq 10^8$ provoquent `OutOfMemoryError`.

4.3 Analyse comparative des performances

4.3.1 Impact de la durée de simulation

Pour évaluer la relation entre la durée de simulation et le temps d'exécution, nous avons mesuré le temps d'exécution pour différentes durées de simulation T avec $\lambda = 5$ et $\mu = 6$ (soit $\rho = 0.833$). Les résultats sont présentés dans la [Figure 9](#).

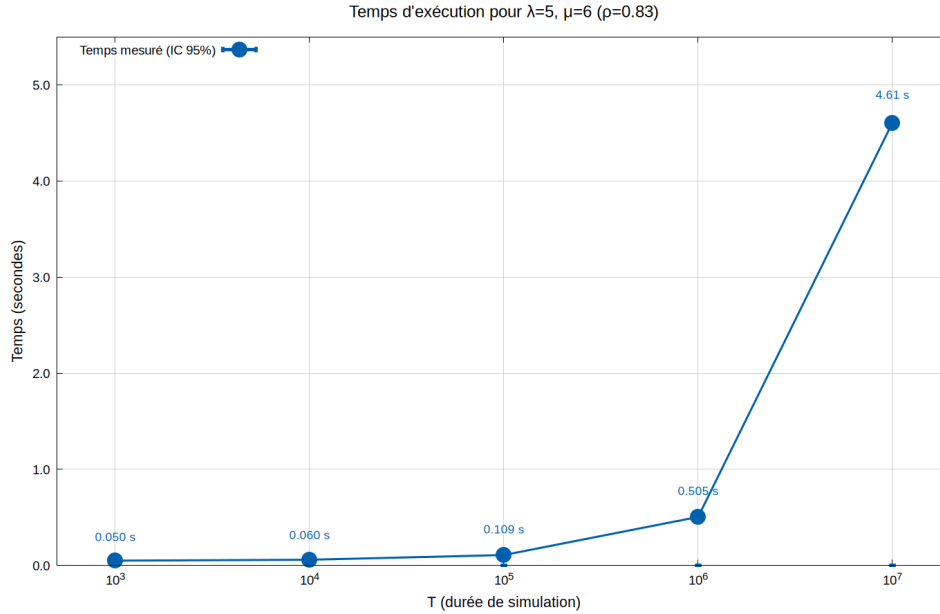


FIGURE 9 – Temps d'exécution en fonction de la durée de simulation T (avec $\lambda = 5$, $\mu = 6$)

Observations :

- **Croissance accélérée** : La courbe montre une croissance légère et presque linéaire entre 10^3 et 10^5 , puis une nette accélération entre 10^5 et 10^6 et enfin une croissance d'autant plus élevée entre 10^6 et 10^7 révélant l'impact de l'accumulation d'événements dans l'échéancier. Le nombre d'événements est proportionnel à T ($(\lambda + \mu) \times T$).
- **Performance** : Le temps d'exécution maximal moyen est de 4.61s pour $T=10^7$, ce qui est dans l'ordre de grandeur attendu pour une simulation de 50 millions de clients.
- **Impact des structures de données** : Pour les petites durées, les surcoûts d'initialisation dominent. Au-delà de 10^5 , le coût d'insertion dans l'échéancier croît avec sa taille, expliquant l'accélération observée.

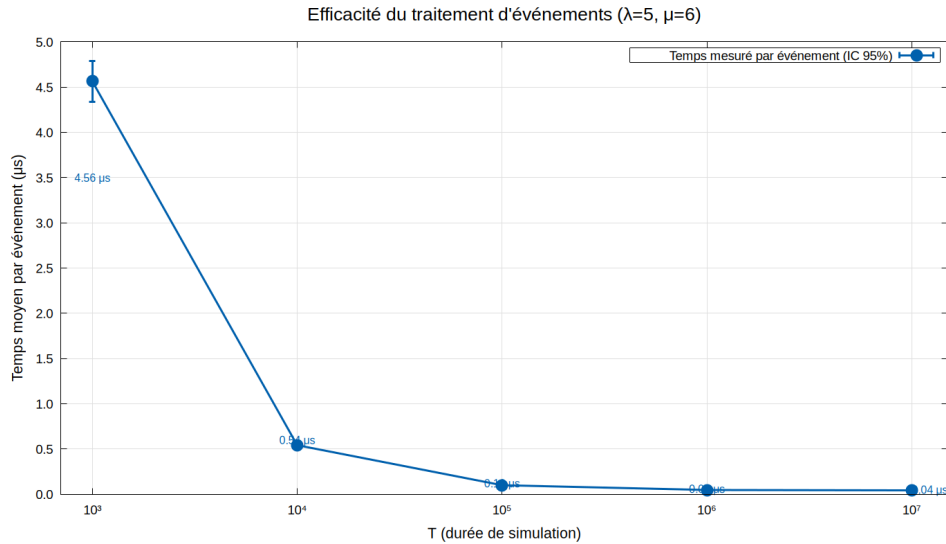


FIGURE 10 – Temps moyen de traitement par événement

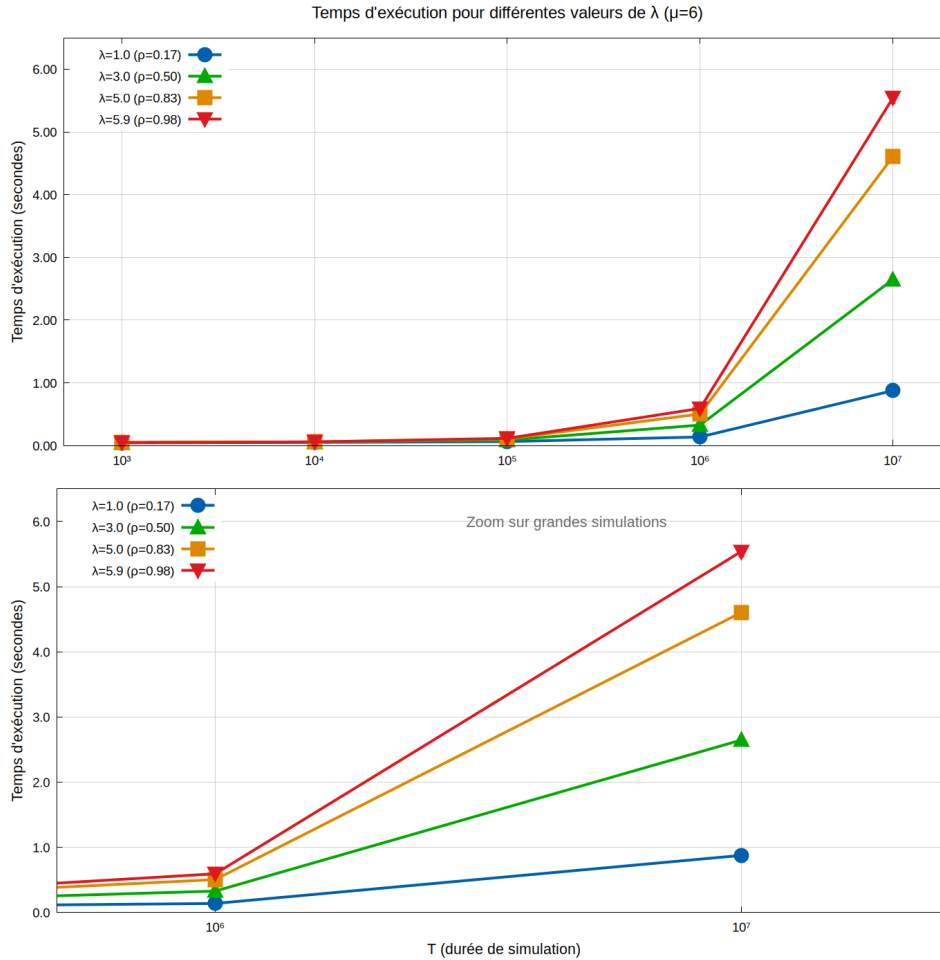
Analyse du temps par événement :

La Figure 10 montre la décroissance du temps moyen de traitement par événement en fonction de la durée de simulation T , avec les intervalles de confiance à 95%.

- **Décroissance rapide** : Le temps par événement chute de 4.6 μs à 10^3 jusqu'à 0.04 μs à 10^7 .
- **Convergence asymptotique** : Pour $T \geq 10^6$, le temps par événement converge vers 0.04-0.05 μs , démontrant que les optimisations (tableau de recyclage, insertion efficace) sont efficaces pour les grandes simulations.
- **Interprétation** : Cette décroissance s'explique par l'amortissement des coûts d'initialisation sur un nombre croissant d'événements. Pour $T=10^3$, on traite environ 11 000 événements ; pour $T=10^7$, on traite environ 110 millions d'événements, soit un coût unitaire beaucoup plus faible grâce à l'amortissement des surcoûts d'initialisation.
- **Note sur les intervalles de confiance** : Les IC sont relativement plus grands pour T petit ($T=10^3$: $\pm 5\%$) car on a moins d'événements et donc moins de données statistiques.

4.3.2 Impact du taux d'arrivée λ

Nous avons également étudié l'impact du taux d'arrivée λ sur le temps d'exécution pour une durée fixe T , comme illustré dans la Figure 11.

FIGURE 11 – Temps d'exécution pour différentes valeurs de λ (durée fixe)**Observations :**

La Figure 11 compare l'impact du taux d'arrivée λ sur les temps d'exécution. Le panel supérieur offre une vue d'ensemble sur toutes les durées, tandis que le panel inférieur zoome sur les grandes simulations.

- **Hiérarchie des temps d'exécution** : Les courbes montrent une nette hiérarchie croissante avec λ : les valeurs faibles ($\lambda = 1.0$, $\rho = 0.17$) génèrent les temps les plus courts, tandis que les valeurs élevées ($\lambda = 5.9$, $\rho = 0.98$) produisent les temps les plus longs. Cette relation s'explique par le nombre d'événements $(\lambda + \mu) \times T$.
- **Croissance accélérée avec la durée** : Le panel supérieur révèle une croissance progressive puis accélérée pour toutes les valeurs de λ . Les petites durées montrent des courbes proches, tandis que les grandes durées ($T \geq 10^6$) révèlent une forte divergence entre les courbes, confirmant l'impact croissant de λ sur les performances.
- **Augmentation de l'écart entre les courbes** : Le panel inférieur (zoom) illustre particulièrement bien l'effet : l'écart entre les différentes valeurs de λ s'accroît significativement entre $T=10^6$ et $T=10^7$, révélant que plus λ se rapproche de μ , plus la complexité du système augmente.
- **Précision statistique** : Les intervalles de confiance (IC à 95%) sont calculés mais pas nécessairement visibles sur les deux panels à cause des échelles différentes et les intervalles assez petits pour ne pas être visibles.

4.4 Conclusion et interprétation croisée

Cette analyse des performances révèle plusieurs aspects complémentaires du simulateur :

Complexité temporelle et rôle des paramètres : L'analyse croisée des [Figures 9-10-11](#) confirme que le temps d'exécution croît essentiellement avec la durée de simulation T et le taux d'arrivée λ . Les optimisations (recyclage d'événements, insertion efficace dans l'échéancier) permettent de maintenir une complexité proche de $O(N)$ même pour de très grandes simulations, mais l'accumulation d'événements dans l'échéancier induit une croissance non linéaire pour les très grandes durées ($T \geq 10^6$).

Performance en fonction de λ : L'impact de λ sur les temps d'exécution révèle un effet croissant avec la durée. Pour $T \geq 10^6$, l'écart entre les différentes valeurs de λ s'amplifie significativement, confirmant que plus le facteur d'utilisation s'approche de 1, plus la complexité de gestion de l'échéancier augmente. Cet effet est particulièrement visible dans le panel zoom de la [Figure 11](#).

Limites pratiques : Malgré les optimisations, les simulations avec $T \geq 10^8$ provoquent des erreurs de mémoire (`OutOfMemoryError`) dues à l'accumulation d'événements dans l'échéancier et au stockage des temps d'arrivée. Cette limite suggère que pour des simulations extrêmement longues, une approche différente serait nécessaire.

Efficacité des optimisations : Les [Figures 9-10](#) montrent que le temps par événement converge vers une valeur très faible ($0.04-0.05 \mu s$ pour $T \geq 10^6$), démontrant l'efficacité des structures optimisées à grande échelle. Les surcoûts d'initialisation dominent pour les petites simulations, mais deviennent négligeables avec l'augmentation de la durée.

5 Synthèse et Réflexions

5.1 Résultats et contributions

Cette étude a permis de confirmer les propriétés théoriques des files M/M/1 par simulation discrète. Les résultats montrent une excellente correspondance entre simulation et théorie pour toutes les métriques, avec des erreurs relatives inférieures à 0.1% sur toute la plage de ρ . Les intervalles de confiance étroits valident la robustesse statistique du simulateur. Les analyses de sensibilité révèlent que les instabilités observées près de la saturation sont des propriétés mathématiques fondamentales du modèle M/M/1, confirmées par l'analyse théorique des dérivées partielles de L et W par rapport à ρ , qui divergent lorsque $\rho \rightarrow 1$.

Les performances obtenues sont satisfaisantes et dans la plage attendue. Les optimisations implémentées (recyclage d'événements, insertion efficace dans l'échéancier) ont permis d'atteindre une complexité proche de $O(N)$, avec un temps par événement convergeant vers $0.04-0.05 \mu s$ pour les grandes simulations. Cependant, l'accumulation d'événements dans l'échéancier induit une croissance non linéaire pour les grandes durées, et les simulations très longues ($T \geq 10^8$) provoquent des erreurs de mémoire (`OutOfMemoryError`).

5.2 Améliorations possibles

Plusieurs améliorations pourraient être envisagées au niveau de l'implémentation :

- **Structure de données optimisée :**
Remplacer la `LinkedList` par une `PriorityQueue` (tas binaire) permettrait d'avoir un coût d'insertion en $O(\log n)$ garantie au lieu de $O(n)$ dans le pire cas pour la `LinkedList`, même si l'optimisation actuelle amortit à $O(1)$ pour les insertions en fin de liste.
- **Gestion mémoire pour simulations longues :** Implémenter un mécanisme de vidage périodique de l'échéancier ou limiter sa taille maximale pour éviter les erreurs `OutOfMemoryError`.
- **Optimisations du tableau de recyclage :** Redimensionner dynamiquement le tableau de recyclage avec croissance exponentielle pour éviter la limitation de taille fixe (`TAILLE_MAX_RECYCLAGE=10000`).

5.3 Apprentissages

Cette implémentation a permis de comprendre que la gestion de l'échéancier est critique pour les performances, et que la validation statistique (IC, jeux) est essentielle pour garantir la robustesse des résultats. Le compromis entre précision et temps de calcul nécessite un arbitrage : simulations longues pour la précision vs optimisations pour les performances.

5.4 Limitations du modèle M/M/1

L'analyse révèle trois catégories de limitations. **Mathématiques :** Le modèle explose lorsque $\rho \rightarrow 1$, avec un seuil critique $\rho = 0.9$ rendant le système inutilisable. **Simulateur :** Les erreurs mémoire apparaissent pour $T \geq 10^8$, et la précision statistique se dégrade fortement pour $\rho > 0.9$ avec une croissance non linéaire des coûts de gestion. **Systèmes réels :** Les hypothèses de capacité infinie, processus Poisson et service exponentiel simplifient excessivement la réalité, excluant la priorisation et les abandons. Le modèle M/M/1 reste adapté pour $\rho < 0.8$, garantissant qualité de service, temps d'attente raisonnables et précision statistique fiable.